APPRENTISSAGE SUPERVISÉ

Algorithmes de Prédiction
A.LArhlimi

Régression & Classification

Mars 2026 · Python · Scikit-learn · XGBoost · LightGBM

Diabetes · California Housing · Iris · Breast Cancer · Wine

19 algorithmes

- Régression Linéaire
- Ridge / Lasso
- Elastic Net
- SVR
- Random Forest
- Gradient Boosting
- MLP
- SVM
- Naive Bayes
- KNN
- Decision Tree
- Logistique

Datasets · Métriques · Code Python · Guide de sélection

Sommaire de la Présentation

01 Introduction — Apprentissage supervisé & familles

02 Métriques d'évaluation (Régression & Classification)

03 Algorithmes de Régression (11 algorithmes)

04 Algorithmes de Classification (8 algorithmes)

05 Tableaux comparatifs globaux

06 Guide de sélection de l'algorithme

07 Bonnes pratiques & Pipeline ML

08 Code Python & Datasets

09 Conclusion

A. Larhlimi

Introduction — Apprentissage Supervisé

Les algorithmes de prédiction apprennent à partir de données étiquetées (paires entrée/sortie) pour prédire la valeur cible de nouvelles observations.

RÉGRESSION

Variable cible continue

- Prix immobilier
- Température
- Score médical
- Chiffre d'affaires
- Évaluation de crédit

CLASSIFICATION

Variable cible catégorielle

- Spam / Non-spam
- Cancer / Sain
- Sentiment positif/négatif
- Chiffre manuscrit (0-9)
- Fraude / Légitime

Métriques d'Évaluation — Régression

Métrique	Formule	Interprétation
RMSE	$\sqrt{(\Sigma(y_i - \hat{y}_i)^2 / n)}$	Erreur moyenne en unité de y. Sensible aux gros écarts.
MAE	$\Sigma\lvert y_i - \hat{y}_i\rvert / n$	Erreur absolue moyenne. Robuste aux outliers.
R^2	$1 - SS_res / SS_tot$	Proportion de variance expliquée. 1 = parfait, 0 = baseline.
MAPE	$\Sigma\lvert y_i - \hat{y}_i\rvert / y_i \times 100$	Erreur relative en pourcentage.

Validation Croisée (K-Fold Cross-Validation)

Diviser les données en k plis → Entraîner sur k−1 plis, tester sur le pli restant → Répéter k fois → Moyenner les scores. Réduit le biais dû au split aléatoire et donne une estimation robuste des performances de généralisation.

Métriques d'Évaluation — Classification

Métrique	Formule	Interprétation
Accuracy	(VP+VN)/(VP+VN+FP+FN)	Taux de bonnes prédictions global.
Précision	VP / (VP + FP)	Parmi les prédits positifs, combien sont vrais.
Rappel	VP / (VP + FN)	Parmi les vrais positifs, combien détectés.
F1-Score	2×(P×R)/(P+R)	Harmonie précision/rappel — utile si classes déséquilibrées.
AUC-ROC	Aire sous la courbe ROC	0.5 = aléatoire, 1 = parfait. Robuste aux classes déséquilibrées.

Matrice de confusion : VP (vrais positifs) · VN (vrais négatifs) · FP (faux positifs) · FN (faux négatifs)
Privilégier F1-Score et AUC-ROC sur des datasets déséquilibrés (fraude, médical).

PARTIE I

Algorithmes de Régression

Régression Linéaire · Ridge · Lasso · Elastic Net · Polynomiale · SVR · KNN · Decision Tree · Random Forest · Gradient Boosting · MLP

1. Régression Linéaire — OLS

Modélise la relation entre X et y par une combinaison linéaire des features. Coefficients estimés par moindres carrés ordinaires (OLS).

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p \qquad \hat{\beta} = (X^T X)^{-1} X^T y$$

✓ Avantages

- Simple, rapide, interprétable
- Coefficients = impact direct de chaque feature
- Intervalles de confiance sur les coefficients
- Pas d'hyperparamètres

✗ Limites

- Suppose une relation strictement linéaire
- Sensible aux outliers et à la multicolinéarité
- Underfitting sur données complexes
- Suppose des résidus normaux

Cas d'usage : Prix immobilier · Modèles économiques · Analyse de régression médicale

2-3. Ridge (L2) & Lasso (L1) — Régularisation

Ridge L2 — $\hat{\beta} = \text{argmin}[SS + \alpha \cdot \Sigma \beta_j^2]$

Réduit les coefficients mais ne les annule jamais. Idéal contre la multicolinéarité.

$$\hat{\beta} = (X^T X + \alpha I)^{-1} X^T y$$

Effet de α :

- $\alpha \to 0$: équivaut à OLS
- $\alpha \to \infty$: coefficients → 0 (jamais exactement 0)
- Choix de α par RidgeCV (cross-validation)

+ Gère la multicolinéarité — Ne sélectionne pas les features

Usage : génomique, finance, features corrélées

Lasso L1 — $\hat{\beta} = \text{argmin}[SS + \alpha \cdot \Sigma |\beta_j|]$

Sélection automatique de features : certains coefficients deviennent exactement zéro.

Losange L1 → coins sur les axes → $\beta_j = 0$

Algorithme :

- Pas de solution analytique (non différentiable en 0)
- Coordinate Descent ou LARS
- Choix de α par LassoCV

+ Modèles parcimonieux — Instable si features corrélées

Usage : sélection de gènes, NLP, haute dimension

4-5. Elastic Net & Régression Polynomiale

Elastic Net — L1 + L2 combinés

$$\alpha \times (\rho \cdot \Sigma|\beta_j| + (1-\rho)/2 \cdot \Sigma\beta_j^2)$$

- α : force globale de régularisation
- ρ (l1_ratio) = 1 → Lasso pur
- ρ = 0 → Ridge pur · 0 < ρ < 1 → compromis
- Gère les features corrélées mieux que Lasso
- Sélection de groupe — 2 hyperparamètres à régler
- Usage : bioinformatique, données médicales

Régression Polynomiale

$$\hat{y} = \beta_0 + \beta_1 x + \beta_2 x^2 + \ldots + \beta_n x^n$$

Degré	Biais	Variance
1	Élevé	Faible → Underfitting
2–3	Moyen	Moyen → Bon équilibre
>5	Faible	Élevée → Overfitting

- Transformation PolynomialFeatures + régression linéaire
- Combinable avec Ridge/Lasso pour régularisation
- Explosion des features avec le degré
- Usage : croissance, physique, relations non-linéaires simples

6. Support Vector Regression (SVR)

Extension de SVM à la régression. Cherche f(x) telle que la majorité des résidus soient dans un tube ε -insensible.

$$\text{Minimiser : } \tfrac{1}{2}\|w\|^2 + C\cdot\Sigma(\xi_i+\xi_i^*) \qquad \text{Sous : } |y_i - f(x_i)| \le \varepsilon + \xi_i$$

Noyau	Formule	Usage
Linéaire	$x^T y$	Données linéairement séparables
RBF (Gaussien)	$\exp(-\gamma\|x-y\|^2)$	Usage général, non-linéaire
Polynomial	$(\gamma x^T y + r)^d$	Relations polynomiales
Sigmoïde	$\tanh(\gamma x^T y + r)$	Similaire aux réseaux de neurones

✓ Avantages

- Efficace en haute dimension
- Robuste aux outliers (vecteurs support seulement)
- Flexible grâce aux noyaux

✗ Limites

- Lent sur grands datasets $O(n^2\text{-}n^3)$
- Choix délicat de C, ε et γ
- Peu interprétable

Cas d'usage : Séries temporelles · Données financières · Bioinformatique

7-8. KNN Régression & Decision Tree Régression

KNN Régression (k=? voisins)

$$\hat{y} = (1/k) \times \Sigma\, y_i \quad \text{(k plus proches voisins)}$$

- Lazy learner : aucun entraînement
- Prédiction : O(n) sans indexation spatiale
- k petit → overfitting · k grand → underfitting
- Choisir k par cross-validation
- Sensible aux features non normalisées
- Malédiction de la dimensionnalité
- Usage : recommandation, interpolation spatiale

Decision Tree Régression

$$\text{Split : minimise MSE} = (1/n)\cdot\Sigma(y_i - \bar{y})^2$$

Paramètre	Effet
max_depth	Profondeur maximale
min_samples_split	Min points pour diviser
ccp_alpha	Élagage (pruning)

- Très interprétable — visualisation directe
- Gère features numériques et catégorielles
- Sans normalisation nécessaire
- Forte tendance à l'overfitting
- Instable (variance élevée)
- Usage : règles métier, base des ensembles

9. Random Forest Régression

Méthode d'ensemble : combine de nombreux arbres de décision indépendants. La prédiction finale = moyenne des prédictions de tous les arbres.

Bootstrap Aggregating (Bagging)

Chaque arbre entraîné sur un sous-échantillon aléatoire avec remise (~63% des données). Les 37% restants = OOB (Out-of-Bag) pour estimer l'erreur gratuitement.

Feature Randomness

À chaque split, seul un sous-ensemble aléatoire de features est considéré (p/3 pour la régression, √p pour la classification). Décolle les arbres et réduit leur corrélation.

Importance des Features

Calculée par la réduction moyenne d'impureté (MDI) apportée par chaque feature à travers tous les arbres. Permet l'interprétation partielle.

✓ Avantages

- Très performant sans réglage fin
- Robuste à l'overfitting (bagging réduit la variance)
- Fournit des importances de features
- Parallélisable — chaque arbre indépendant

✗ Limites

- Moins interprétable qu'un seul arbre
- Lent sur très grands datasets
- Mauvais en extrapolation (hors plage d'entraînement)
- Mémoire élevée (nombreux arbres)

Cas d'usage : prédiction de prix · détection de fraudes · analyse médicale · tout problème tabulaire

10. Gradient Boosting — GBM, XGBoost, LightGBM

Construit des arbres séquentiellement : chaque arbre corrige les résidus du précédent. Parmi les meilleurs sur données tabulaires.

$$F_m(x) = F_{m-1}(x) + \eta \times h_m(x) \qquad r_{im} = -\partial L(y_i, F(x_i)) / \partial F(x_i)$$

Algorithme	Caractéristiques clés
GBM sklearn	Référence, pur Python, relativement lent
XGBoost	Régularisation L1/L2, parallélisme, pruning, valeurs manquantes
LightGBM	Leaf-wise, GOSS, EFB — ultra-rapide sur grands datasets
CatBoost	Variables catégorielles natives, ordered boosting

✓ Avantages

- Meilleures perfs sur données tabulaires
- Gère les valeurs manquantes (XGBoost, LightGBM)
- Robuste aux outliers (perte Huber)

✗ Limites

- Nombreux hyperparamètres à régler
- Entraînement séquentiel (difficile à //)
- Overfitting si n_estimators élevé sans early stopping

Cas d'usage : Compétitions ML (Kaggle) · Finance · Fraudes · Prédiction d'churn

11. MLP Régression — Réseau de Neurones

Multi-Layer Perceptron : apprend une fonction non-linéaire complexe en composant des transformations linéaires et des activations non-linéaires.

$$h^{(l)} = \sigma(W^{(l)} h^{(l-1)} + b^{(l)})$$

Perte MSE → Backprop → SGD/Adam

Activation	Formule	Usage
ReLU	max(0, x)	Couches cachées (défaut)
tanh	$(e^x - e^{-x})/(e^x + e^{-x})$	Couches cachées
Sigmoid	$1/(1+e^{-x})$	Sortie classif. binaire
Linéaire	x	Sortie régression

✓ Avantages

- Approximateur universel (n'importe quelle fonction)
- Très performant sur données volumineuses
- Capture des interactions très complexes
- Flexible en architecture

✗ Limites

- Boîte noire — peu interprétable
- Nécessite beaucoup de données et de calcul
- Nombreux hyperparamètres
- Sensible à l'initialisation et normalisation

Cas d'usage : Séries temporelles · Données multimodales · Approximation de fonctions complexes

PARTIE II

Algorithmes de Classification

Régression Logistique · KNN · Naive Bayes · SVM · Decision Tree · Random Forest · Gradient Boosting · MLP

12. Régression Logistique

Malgré son nom, c'est un algorithme de classification. Modélise P(y=1|x) via la fonction sigmoïde. Frontière de décision linéaire.

$$P(y=1|x) = \sigma(w^T x+b) = 1/(1+e^{-(w^T x+b)}) \qquad L = -(1/n)\cdot\Sigma[y_i \log \hat{y}_i + (1-y_i) \log(1-\hat{y}_i)]$$

Extension multi-classe

One-vs-Rest (OvR) : K classificateurs binaires
Softmax (multinomial) : modélise directement P(y=k|x) pour K classes.

Régularisation

L1 (Lasso) → sélection de features
L2 (Ridge) → réduction des coefficients
Combinables via hyperparamètre C = 1/α.

AUC-ROC

Courbe TPR vs FPR en variant le seuil de décision. AUC = 0.5 (aléatoire), AUC = 1 (parfait). Robuste aux déséquilibres de classes.

✓ Avantages

- Interprétable (coefficients = log-odds)
- Fournit des probabilités calibrées
- Rapide à entraîner
- Bonne baseline classification

✗ Limites

- Frontière de décision linéaire uniquement
- Sensible aux features non normalisées
- Underfitting sur données non-linéaires
- Hypothèse linéaire restrictive

Cas d'usage : Scoring de crédit · Diagnostic médical · Détection de spam

13-14. KNN Classification & Naive Bayes

KNN Classification — Classe majoritaire

Métrique	Formule / Usage
Euclidienne	$\sqrt{(\Sigma(x_i-y_i)^2)}$ — données continues
Manhattan	$\Sigma\lvert x_i-y_i\rvert$ — sparses, discrètes
Cosinus	$1-(x\cdot y)/(\lVert x\rVert\lVert y\rVert)$ — texte/embed.

- k=1 : frontière Voronoï, sensible au bruit
- k impair pour éviter les égalités (binaire)
- Apprentissage instantané — lazy learner
- Prédiction O(n) — lent · Mémoire élevée
- Malédiction de la dimensionnalité
- Usage : recommandation, vision, anomalies

Naive Bayes — Théorème de Bayes

$$P(y|x_1...x_p) \propto P(y) \times \Pi\ P(x_j|y)$$

Variante	Usage
GaussianNB	Features continues
MultinomialNB	Comptage de mots (NLP)
BernoulliNB	Features binaires

- Hypothèse : features indépendantes (naïve)
- Extrêmement rapide — entraînement et prédiction
- Fonctionne bien avec peu de données
- Très bon en NLP (MultinomialNB)
- Hypothèse rarement vérifiée en pratique
- Usage : anti-spam, classification de docs

15. Support Vector Machine (SVM)

Cherche l'hyperplan à marge maximale séparant les classes. Les vecteurs support définissent entièrement la solution.

$$\text{Maximiser } 2/\|w\| \quad \text{Sous : } y_i(w^T x_i + b) \geq 1 \qquad \text{Soft Margin : } \min \tfrac{1}{2}\|w\|^2 + C \cdot \Sigma \xi_i$$

Vecteurs Support

Points les plus proches de l'hyperplan. Définissent entièrement la solution — tous les autres points n'ont aucun impact. Efficacité mémoire.

Paramètre C

C grand → faible régularisation → risque overfitting.
C petit → forte régularisation → plus de points mal classés tolérés.

Kernel Trick

$K(x,y) = \phi(x)^T\phi(y)$. Projette implicitement dans un espace de grande dim. sans calculer les coordonnées. Noyaux : RBF, Polynomial, Linéaire.

✓ Avantages

- Très performant en haute dimension
- Efficace en mémoire (vecteurs support)
- Théoriquement bien fondé
- Flexible grâce aux noyaux

✗ Limites

- Lent sur grands datasets $O(n^2\text{-}n^3)$
- Sensible au choix du noyau et de C
- Peu interprétable
- Mauvaise gestion des classes déséquilibrées

Cas d'usage : Classification d'images · Bioinformatique · Reconnaissance de texte

16. Decision Tree Classification

Partitionne l'espace des features récursivement selon des règles maximisant la pureté des noeuds. Très interprétable.

Gini Impurity

$$Gini = 1 - \Sigma\, p_i^2$$

Entropie (Information Gain)

$$H = -\Sigma\, p_i \log_2(p_i)$$

Algorithme	Critère	Splits
CART	Gini ou MSE	Binaires
ID3	Information Gain	Multiples
C4.5	Gain ratio	Multiples

Pruning (Élagage)

Pre-pruning : limiter profondeur, noeuds, feuilles.
Post-pruning : construire complet puis élager (ccp_alpha).

✓ Avantages

- Visualisable et interprétable
- Features numériques et catégorielles
- Pas de normalisation nécessaire
- Capture les interactions

✗ Limites

- Overfitting facile (sans contrainte)
- Instable (variance élevée)
- Biais vers features à nombreuses modalités
- Frontières rectangulaires

17-18. Random Forest & Gradient Boosting Classification

Random Forest — Vote majoritaire

$$\text{Erreur} \leq \bar{\rho} \times (1-s^2)/s^2$$
(ρ=corrélation, s=force)

OOB Error : ~37% des données non vues par chaque arbre → estimation gratuite de l'erreur de généralisation.

- Vote majoritaire des arbres
- Feature importance fiable (MDI)
- Très robuste à l'overfitting
- Gère les valeurs manquantes (OOB)
- Importance des features interprétable
- Usage : fraudes, médical, scoring

AdaBoost · XGBoost · LightGBM

$$F(x) = \Sigma\, \alpha_m\, h_m(x)$$ Log-Loss / Softmax

AdaBoost : weak learners séquentiels — poids augmentés sur les mal classés. XGBoost : +L1/L2, pruning, valeurs manq. LightGBM : leaf-wise, GOSS, EFB — ultra-rapide.

- Meilleures perfs sur données tabulaires
- Early stopping contre l'overfitting
- Gestion native des valeurs manquantes
- Nombreux hyperparamètres à régler
- Vainqueurs de compétitions Kaggle
- Usage : scoring bancaire, fraudes

19. MLP Classification — Réseau de Neurones

Couche de sortie adaptée : Sigmoid (binaire) ou Softmax (multi-classe). Fonctions de perte : Binary ou Categorical Cross-Entropy.

$$P(y=k|x) = \exp(z_k) / \Sigma_j \exp(z_j)$$

$$\text{Categorical Cross-Entropy} = -\Sigma\, y_i \log \hat{y}_i$$

Dropout

Désactiver aléatoirement des neurones pendant l'entraînement. Réduit la co-adaptation des neurones.

L2 Weight Decay

Pénaliser les poids trop grands. Analogue à Ridge pour les réseaux.

Early Stopping

Arrêter l'entraînement quand la validation loss ne s'améliore plus. Simple et efficace.

Batch Normalization

Normaliser les activations inter-couches. Stabilise et accélère l'entraînement.

✓ Avantages

- Capture des patterns très complexes
- Flexible (profondeur, largeur, activations)

✗ Limites

- Boîte noire — difficile à interpréter
- Beaucoup de données et de calcul requis

Cas d'usage : Vision par ordinateur : NLP : Classification médicale : Transformers

VUE D'ENSEMBLE

Tableaux Comparatifs & Benchmarks

Tableau Comparatif — Algorithmes de Régression

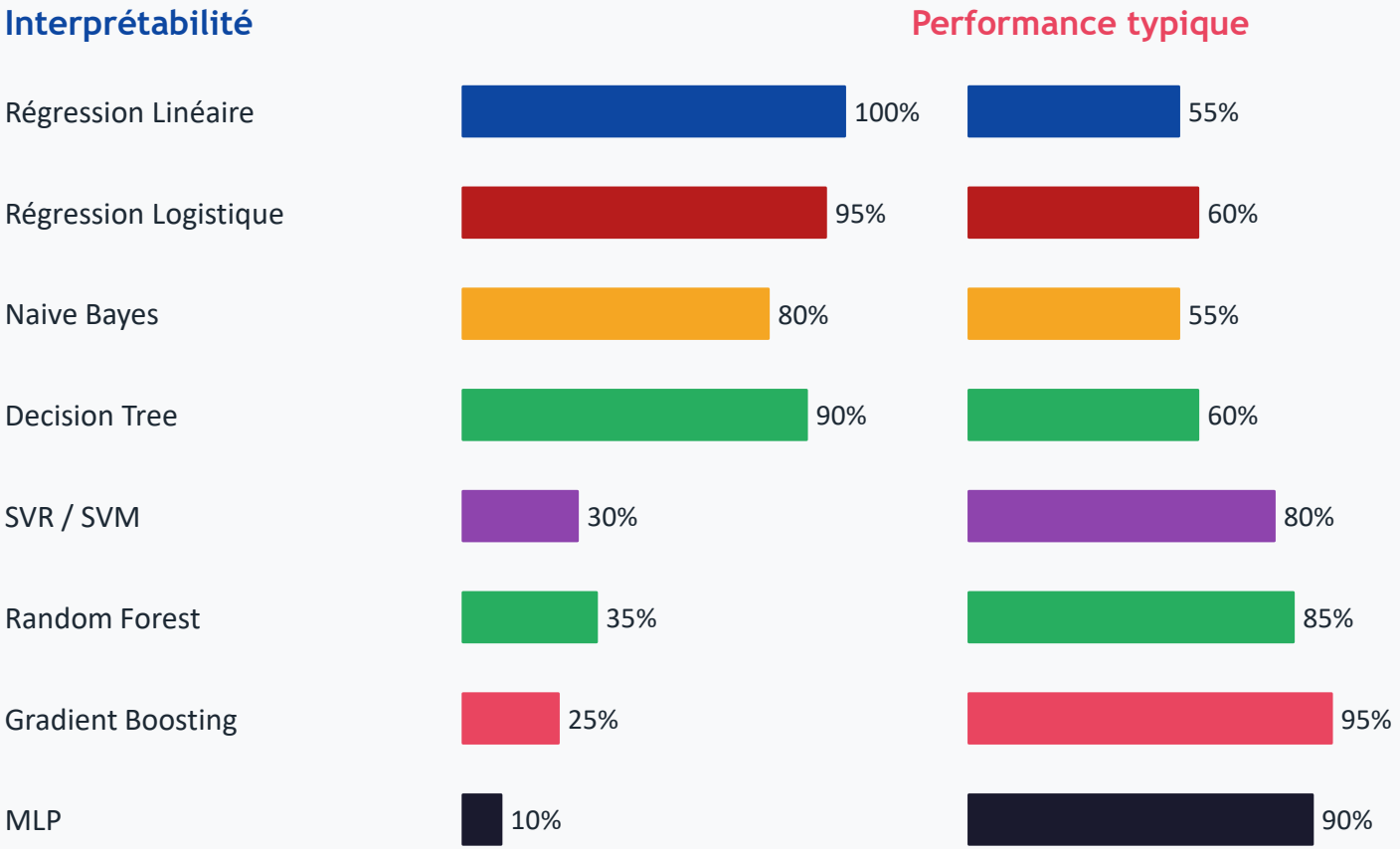
Algorithme	Linéaire	Robustesse outliers	Scalabilité	Interprét.
Régression Linéaire	Oui	Faible	Très haute	Très haute
Ridge	Oui	Faible	Très haute	Haute
Lasso	Oui	Faible	Haute	Haute (sparse)
Elastic Net	Oui	Faible	Haute	Haute
Polynomiale	Non	Faible	Moyenne	Moyenne
SVR	Non	Élevée	Faible	Faible
KNN	Non	Faible	Faible	Faible
Decision Tree	Non	Moyenne	Haute	Très haute
Random Forest	Non	Élevée	Haute	Faible
Gradient Boosting	Non	Élevée	Haute	Faible
MLP	Non	Moyenne	Très haute	Très faible

Tableau Comparatif — Algorithmes de Classification

Algorithme	Frontière	Probabilités	Scalabilité	Interprét.
Régression Logistique	Linéaire	Oui (calibrées)	Très haute	Très haute
Naive Bayes	Linéaire	Oui	Très haute	Haute
KNN	Non-linéaire	Oui	Faible	Faible
SVM	Non-linéaire	Non*	Faible	Faible
Decision Tree	Non-linéaire	Oui	Haute	Très haute
Random Forest	Non-linéaire	Oui	Haute	Faible
AdaBoost	Non-linéaire	Oui	Haute	Faible
Gradient Boosting	Non-linéaire	Oui	Haute	Faible
MLP	Non-linéaire	Oui	Très haute	Très faible

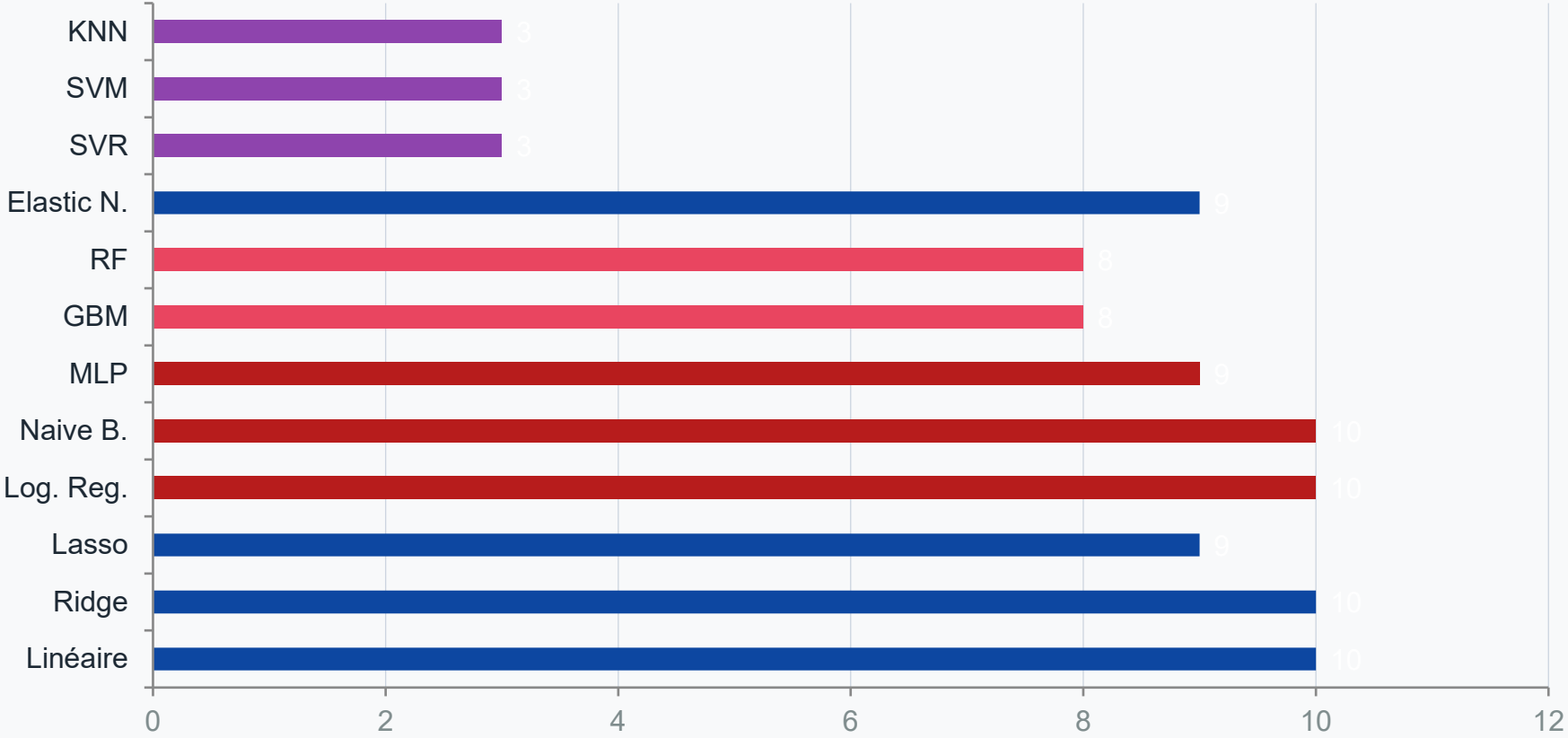
** SVM peut fournir des probabilités via Platt scaling (probability=True).*

Comparaison : Interprétabilité vs Performance



A. Larhlimi

Comparaison : Scalabilité & Complexité



GUIDE PRATIQUE

Choisir le Bon Algorithme

Guide de Sélection — Régression

Relation linéaire ?

Ridge / Elastic Net (features corrélées)
Lasso (sélection features)
Linéaire (cas simple)

Non-linéaire : taille du dataset ?

Petit/Moyen → SVR
Tabulaire → Random Forest / Gradient Boosting
Grands données → MLP

Cas particuliers

Haute dimension (p >> n)

Ridge · Lasso · Elastic Net · MLP

Données très bruitées / outliers

SVR · Random Forest · Gradient Boosting (perte Huber)

Interprétabilité requise

Linéaire · Ridge · Lasso · Decision Tree

Règle d'or : toujours commencer par une régression linéaire/Ridge comme baseline.

A. Larhlimi

Guide de Sélection — Classification

Interprétabilité importante

→ Régression Logistique · Decision Tree · Naive Bayes

Petits datasets (<1000 échant.)

→ SVM · Naive Bayes · KNN · Logistique

Grands datasets (>10K)

→ Gradient Boosting · MLP · Random Forest

Haute dimension (texte, images)

→ SVM · MLP · Logistique + L1

Données déséquilibrées

→ Gradient Boosting + class_weight · SMOTE

Probabilités calibrées requises

→ Régression Logistique · Naive Bayes · Calibrated SVM

Règle d'or : Gradient Boosting (XGBoost/LightGBM) = meilleur choix par défaut sur données tabulaires.

A. Larhlimi

BONNES PRATIQUES

Pipeline ML & Conseils Pratiques

Pipeline ML Recommandé

Exploration (EDA)

01

Distributions, corrélations, valeurs manquantes. Visualisations PCA/scatter. Comprendre les données avant tout.

Préprocessing

02

Normalisation (StandardScaler pour SVM/KNN/MLP). Encodage des variables catégorielles. Imputation des manquants.

Baseline

03

Régression linéaire / logistique. Naive Bayes. Établir un benchmark simple à battre.

Modèles complexes

04

Random Forest d'abord (robuste, peu de réglage). Puis Gradient Boosting (XGBoost/LightGBM).

Optimisation

05

GridSearchCV ou RandomizedSearchCV. Optuna pour la recherche bayésienne. Optimiser les hyperparamètres clés.

Validation & Interp.

06

Cross-validation stratifiée. Learning curves. SHAP values. Permutation importance.

Règles d'Or en Machine Learning

Toujours normaliser	Pour SVM, KNN, Régression Logistique, MLP — ces algorithmes sont très sensibles à l'échelle des features. StandardScaler ou MinMaxScaler systématiquement.
Cross-valider avant de conclure	Un seul train/test split est insuffisant. K-Fold CV (k=5 ou 10) pour une estimation robuste. Stratified pour les classes déséquilibrées.
Commencer simple	Une bonne baseline (linéaire/logistique) évite de suringéniérer. Si le modèle simple est déjà bon, inutile de complexifier.
Gradient Boosting par défaut	XGBoost ou LightGBM sont souvent les meilleurs choix sur des données tabulaires. Point de départ recommandé pour les compétitions.
MLP : beaucoup de données	MLP nécessite beaucoup de données pour surpasser Gradient Boosting. Sur des petits datasets, privilégier les méthodes classiques.

Pièges & Erreurs Fréquentes

⚠ **Data Leakage**	Laisser de l'information future dans les données d'entraînement. Entraîne des perfs artificiellement élevées qui ne se généralisent pas. Encoder/scaler APRÈS le split.
⚠ **Suroptimisation du dataset de test**	Tester de nombreux modèles sur le même test set "contamine" les résultats. Utiliser un ensemble de validation séparé ou de la cross-validation.
⚠ **Ignorer le déséquilibre des classes**	Accuracy = 99% avec 99% négatifs n'est pas un bon modèle. Utiliser F1-Score, AUC-ROC. Techniques : class_weight, SMOTE.
⚠ **Comparer sans même métrique**	Comparer accuracy vs F1 vs AUC sur des datasets différents invalide toute comparaison. Standardiser le protocole d'évaluation.
⚠ **Overfitting des hyperparamètres**	Optimiser les hyperparamètres sur le test set conduit à de l'overfitting. Utiliser GridSearchCV avec CV interne.

IMPLÉMENTATION

Datasets de Référence & Code Python

Datasets de Référence — Benchmarks

Diabetes

Régression

10 features

442 patients

Cible : Score maladie

California Housing

Régression

8 features

20640 obs.

Cible : Prix médian

Iris

Classification

4 features

150 fleurs

Cible : 3 espèces

Breast Cancer

Classification

30 features

569 tumeurs

Cible : Bénin/Malin

Wine

Classification

13 features

178 vins

Cible : 3 origines

Code Python — Régression (Pipeline complet)

```python
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import cross_val_score
from sklearn.metrics import r2_score, mean_squared_error

# Pipeline : normalisation + modèle
pipe = Pipeline([
    (
'scaler', StandardScaler()),
    (
'model', RandomForestRegressor(n_estimators=100, random_state=42))
])

# Cross-validation
scores = cross_val_score(pipe, X, y, cv=5, scoring='r2')
print(f"R² CV : {scores.mean():.4f} ± {scores.std():.4f}")

# Entraînement final
pipe.fit(X_train, y_train)
y_pred = pipe.predict(X_test)
print(f"R² test : {r2_score(y_test, y_pred):.4f}")
```

Code Python — Classification (XGBoost + Metrics)

```python
import xgboost as xgb
from sklearn.metrics import (
    classification_report,
    roc_auc_score
)

# Modèle XGBoost
model = xgb.XGBClassifier(
    n_estimators=
300,
    learning_rate=
0.05,
    max_depth=
6,
    use_label_encoder=
False,
    eval_metric=
'logloss'
)
model.fit(
    X_train, y_train,
    eval_set=[(X_val, y_val)],
    early_stopping_rounds=
20,
    verbose=
False
)

# Métriques
y_pred = model.predict(X_test)
print(classification_report(y_test, y_pred))
```

```python
from sklearn.model_selection import
    StratifiedKFold, cross_validate

# CV stratifiée
skf = StratifiedKFold(n_splits=5,
    shuffle=
True, random_state=42)

results = cross_validate(
    model, X, y,
    cv=skf,
    scoring=[
'accuracy',

'f1_macro',

'roc_auc_ovr']
)

# AUC-ROC
y_proba = model.predict_proba(X_test)
auc = roc_auc_score(
    y_test, y_proba,
    multi_class=
'ovr'
)
print(
f"AUC: {auc:.4f}")
```

Code Python — LightGBM + Interprétabilité SHAP

```python
import lightgbm as lgb
import shap

# LightGBM avec early stopping
model = lgb.LGBMClassifier(
    n_estimators=
500, learning_rate=0.03,
    num_leaves=
31, subsample=0.8,
    colsample_bytree=
0.8, random_state=42
)
model.fit(X_train, y_train,
    eval_set=[(X_val, y_val)],
    callbacks=[lgb.early_stopping(
50), lgb.log_evaluation(100)])

# Interprétabilité SHAP
explainer = shap.TreeExplainer(model)
shap_values = explainer.shap_values(X_test)

# Visualisation des importances
shap.summary_plot(shap_values, X_test, plot_type="bar")  # Feature importance globale
shap.waterfall_plot(shap_values[0])   # Explication individuelle
```

Conclusion

L'apprentissage supervisé offre une palette d'algorithmes allant des modèles simples et interprétables aux modèles complexes et très performants. Le choix optimal dépend des données, des contraintes et des objectifs.

Gradient Boosting

Référence sur données tabulaires. XGBoost/LightGBM = point de départ recommandé pour la plupart des problèmes supervisés.

Réseaux de Neurones

Dominent sur les données non structurées (images, texte, audio). Nécessitent beaucoup de données et de calcul.

Modèles Linéaires

Indispensables comme baseline et quand l'interprétabilité est cruciale. Ridge/Lasso pour la régression, Logistique pour la classification.

Validation Rigoureuse

Cross-validation, métriques adaptées (F1, AUC-ROC pour classes déséquilibrées), et validation du protocole sont les piliers d'une démarche ML solide.

Algorithmes de Prédiction

Apprentissage Supervisé — Régression & Classification

RÉGRESSION — 11 algorithmes

- Régression Linéaire
- Ridge (L2)
- Lasso (L1)
- Elastic Net
- Polynomiale
- SVR
- KNN
- Decision Tree
- Random Forest
- Gradient Boosting

CLASSIFICATION — 8 algorithmes

- Régression Logistique
- KNN Classification
- Naive Bayes
- SVM
- Decision Tree
- Random Forest
- AdaBoost / XGBoost
- LightGBM / MLP